Transporte Escolar · Frontend

Estado actual del frontend (Vue) al 10 de noviembre de 2026 · rama `Ernest-dev` · último commit `3120b8e` (6 nov 2026) · el árbol de trabajo tiene un cambio sin commitear (una clase de Tailwind en `VisorCaptura.vue`: `z-[70]` → `z-70`).

Documento para quien necesite entender el diseño sin leer el código. Cada dato se comprobó contra el código fuente del repositorio; lo que no se pudo comprobar está marcado no verificado. Apoyos: `README.md` y `docs/`. No incluye credenciales, valores del `.env` ni datos de familias.

> **Comprobado hoy:** `node scripts/verificar-arquitectura.mjs` y con `--estricto`: 105 archivos, 0 fallos, 0 pendientes (exit 0). `npx vue-tsc --noEmit`: sin errores. **No ejecutado hoy:** `npm run build`, linters, capturas y pruebas de comportamiento (necesitan la API y navegador); sus resultados que se citan vienen de los informes en `docs/` (fecha de las fases: nov 2026).

1. Qué hace el sistema y sus pantallas

Panel administrativo web para un servicio de transporte escolar: clientes (alumnos y sus tutores), prospectos, pagos semanales, asistencias y conductores. Es una SPA Vue 3 + TypeScript + Tailwind CSS v4 + Pinia + Vue Router 5 + Vite 8 + axios, sin pruebas unitarias propias (ver §7). Consume una API REST en Go de un repositorio independiente (README; el código del backend no verificado, no está aquí).

Pantalla	Qué hace (según su código)
Login	Correo y contraseña; guarda el token. Pública.
Registro público	Formulario público de alta de prospecto: consulta teléfono, lista escuelas públicas, `POST /prospectos`.
Dashboard	Resumen (`GET /dashboard`) con contadores por estatus de pago y casos especiales; salta a Pagos con filtros mediante un loader de transición.
Clientes	Lista agrupada por tutor, búsqueda, filtro de escuela; modal de detalle, edición, archivar/reincorporar, hermanos, panel de dinero/adeudo.
Prospectos	Lista agrupada por tutor; detalle, edición, descartar/restaurar, convertir en cliente; actualización en tiempo real (SSE).
Pagos	Semana seleccionable, alumnos por escuela y estatus; confirmar/rechazar, marcar pagado, abonos por familia, anular recibos, configurar cuota; SSE.
Casos especiales	Lista por semana y registro de un pago de caso especial (`POST /clientes/{id}/pagos/caso-especial`), con diálogo de confirmación. Cuelga del menú como hijo de Pagos.
Familias	Familias (alumnos agrupados por tutor); alta de hermano vía `agregarAlumno`; consulta de teléfono.
Asistencias	Asistencia del día por escuela (`GET /asistencias`, `PUT /asistencias/{id}`); recuerda la escuela elegida.
Conductores	Lista (con SSE), perfil, ruta/paradas; sub-pantallas «Sus alumnos» y «Agregar alumnos» (asignaciones en lote).
Apariencia	Color de acento y fuente (`PATCH /configuracion`). El enlace del menú solo se muestra si el rol es `admin`.

El marco autenticado (`AppLayout`) muestra el menú: Dashboard, Clientes, Prospectos, Pagos, Casos especiales (sub-ítem), Familias, Asistencias, Conductores; más el acceso a Apariencia (solo admin) y el conmutador claro/oscuro. Los detalles finos de cada pantalla (campos exactos, reglas de negocio) no verificados más allá de lo anterior.

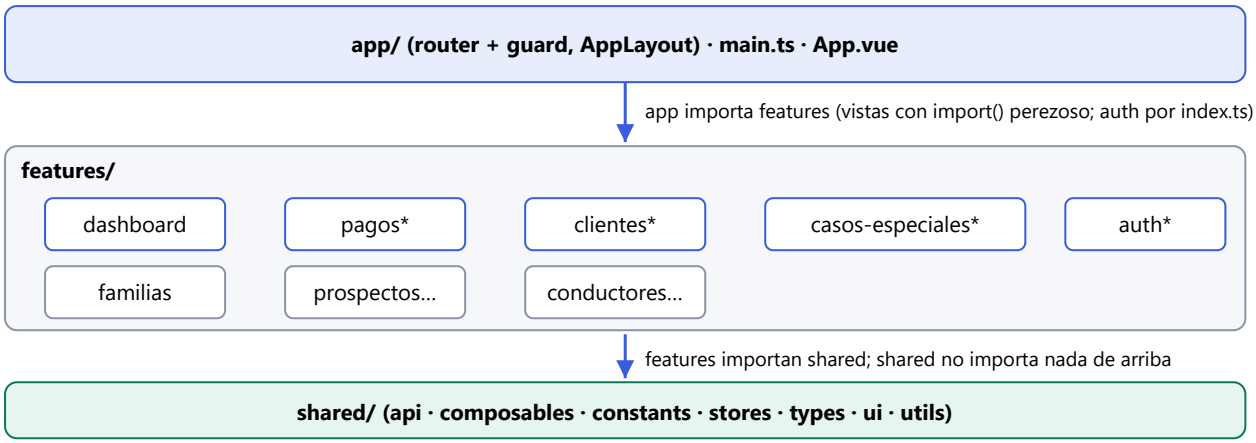
2. Arquitectura

El código en `src/` se divide en tres zonas (el alias `@` apunta a `src`):

- **app/**: `router/index.ts` (rutas + guard) y `layouts/AppLayout.vue`. En la raíz de `src/` quedan `main.ts`, `App.vue` y `assets/main.css`.
- **features/** (11): auth, clientes, prospectos, pagos, familias, casos-especiales, asistencias, conductores, dashboard, apariencia, registro-publico. Cada una con solo las carpetas que necesita: `api/ types/ components/ utils/`

`stores/ views/`.

- **shared/**: `api/ composables/ constants/ stores/ types/ ui/ utils/`; lo que usan varios features. No depende de ninguno.



* = features con `index.ts` (barrel). Aristas aprobadas entre features: dashboard→pagos, dashboard→casos-especiales, pagos→clientes, pagos→auth, clientes→auth, familias→clientes, casos-especiales→clientes (las 7 que el verificador detecta hoy).

Reglas de dependencia (R1–R8, las aplica el verificador)

Regla	Contenido
R1	`shared` no importa de ningún feature, de `app` ni de `auth` (por eso `http.ts` maneja el 401 por su cuenta).
R2	Un feature solo importa de otro por su `index.ts` y solo por las aristas aprobadas (`ARISTAS_APROBADAS` en `scripts/arquitectura/mapa.mjs`).
R3	Ningún feature importa de `app`.
R4	Las vistas `*View.vue` nunca se importan estáticamente; solo `app/router` las carga con `import()`.
R5	Un `index.ts` reexporta solo con nombre (sin `export *`) y nunca archivos `.vue`.
R6	Un archivo no importa el `index.ts` de su propio feature.
R7	Sin ciclos entre archivos ni entre features.
R8	Todo import (`@/` o relativo) resuelve a un archivo existente.

Política de barrels

- Solo se crea `features/<x>/index.ts` cuando otro feature necesita algo de `<x>`; hoy: `auth` (`useAuthStore`), `clientes` (`listarClientes, agregarAlumno, configurarCliente` + 3 tipos), `pagos` (`verPagosSemana, ETIQUETAS, ETIQUETAS_METODO, estaAtenuado` + 2 tipos), `casos-especiales` (`listarCasosEspeciales` + tipo `CasoEspecial`).
- Exportan lo mínimo que consume el otro feature; nunca componentes (para no arrastrarlos al chunk de otra vista).
- Va en `shared` lo que usan dos o más features sin pertenecer a ninguno; lo de un solo feature se queda allí aunque sea genérico. No se crea un componente «genérico» para un único uso.
- Si dos features se necesitan mutuamente, lo común sube a `shared`; un barrel no lo resuelve.

El verificador

`scripts/verificar-arquitectura.mjs` (solo lectura, sin dependencias): recorre `src/**/*.ts|vue`, extrae los imports (ignorando comentarios), arma el grafo y comprueba R1–R8, incluyendo detección de ciclos (Tarjan). Sale con código 1 si hay fallos. `--detalle` lista pendientes; `--estricto` los convierte en fallos. Usa `scripts/arquitectura/mapa.mjs` (mapa de movimientos del refactor, aristas aprobadas, features con barrel). Hoy: 0 fallos.

3. Rutas del router

Historia HTML5 (`createWebHistory(import.meta.env.BASE_URL)`). Todas las vistas se cargan con `import()` perezoso. Las rutas hijas de `/` exigen autenticación (`meta.requiresAuth`). No hay ruta comodín (`*`) ni redirecciones declaradas.

Path	name	Feature · vista	Acceso
/login	login	auth · LoginView	pública (si ya hay sesión redirige a `home`)
/registro	registro-publico	registro-publico · RegistroPublicoView	pública
/	home	dashboard · DashboardView (dentro de AppLayout)	auth
/clientes	clientes	clientes · ClientesView	auth
/clientes/:id	cliente-detalle	clientes · ClientesView (abre el modal de detalle)	auth
/clientes/:id/editar	cliente-editar	clientes · ClientesView (modal en modo edición)	auth
/prospectos	prospectos	prospectos · ProspectosView	auth
/prospectos/:id	prospecto-detalle	prospectos · ProspectosView	auth
/pagos	pagos	pagos · PagosView	auth
/conductores	conductores	conductores · ConductoresView	auth
/conductores/:id	conductor-detalle	conductores · ConductoresView	auth
/conductores/:id/alumnos	conductor-alumnos	conductores · ConductorAlumnosView	auth
/conductores/:id/agregar	conductor-agregar	conductores · ConductorAgregarView	auth
/apariencia	apariencia	apariencia · ApariencíaView	auth (el enlace del menú solo para `admin`; la ruta en sí no comprueba rol)
/familias	familias	familias · FamiliasView	auth
/asistencias	asistencias	asistencias · AsistenciasView	auth
/casos-especiales	casos-especiales	casos-especiales · CasosEspecialesView	auth

Guard global (`beforeEach`): sin sesión en ruta protegida → `login`; con sesión en `login` → `home`; si cambia el primer segmento del path (`clientes`, `pagos` …) muestra el loader de pantalla (`navegacion.mostrar()`); `afterEach` lo oculta. Abrir/cerrar un modal dentro de la misma sección no dispara el loader. «Autenticado» = existe un token en la store (no se valida su vigencia en el cliente; eso lo hace el 401).

4. Cómo lee el frontend del backend

```
Vista (features/*/views)
 └─ [composable o store, solo si hay lógica compartida]
     └─ función de api (features/*/api o shared/api)
         └─ shared/api/http.ts  (axios, baseURL = VITE_API_URL)
             ├─ request: añade  Authorization: Bearer <token>
             ├─ 2xx + JSON → la función devuelve data → la vista lo guarda en un ref
             ├─ 401        → borra "token", window.location.href = '/login'
             └─ otro error → la vista lo captura → shared/utils/errores.ts → mensaje
```

- **Cliente único:** `shared/api/http.ts` crea la instancia axios con `baseURL = import.meta.env.VITE_API_URL`. Las funciones de `api/` usan `http.get/post/put/patch`, devuelven `data` y normalizan (p. ej. listas `null` → `[]`). No hay `delete` en uso (verificado por búsqueda).
- **Token:** el interceptor de request lee `localStorage['token']` en cada petición (no lo guarda en memoria) y, si existe, pone el encabezado Bearer. La store `auth` lo escribe en `login()` (`POST /login` → `{token}`) y lo borra en `logout()`; decodifica el payload del JWT (`atob`, sin verificar firma) solo para leer `rol`.
- **El 401:** el interceptor de respuesta borra `token` y hace `window.location.href = '/login'` (recarga completa, fuera del router) y luego rechaza el error igualmente, así que el `catch` de la vista también corre.
- **Errores en texto plano:** el servidor explica sus errores como texto en el cuerpo (`error.response.data` es `string`). `shared/utils/errores.ts`: `mensajeDeError(e, respaldo)` (texto del servidor, recortado, o el respaldo), `mensajeDeEstado(e, estado, respaldo)` (igual pero solo para ese código), `textoDeEstado` (texto o `null`), `esEstado`, `esEstadoConTexto`. Cada pantalla pone su respaldo en español. Las cargas de listas muestran un mensaje fijo; el texto del servidor se muestra en formularios y diálogos de escritura. Excepción documentada: el 409 al reincorporar con deuda trae un objeto con `deuda` y lo lee `ClienteDetalleModal` **(lectura directa no revisada línea a línea)**. Caso especial: un 409 cuyo texto contiene «ya se registró» en el alta de abono se trata como éxito (doble toque).
- **SSE:** `shared/api/eventos.ts` → `escucharEventos(ruta, alEvento, senal)` usa `fetch` (no axios) porque `EventSource` no permite `Authorization`; URL = `http.defaults.baseURL + ruta`; lee el flujo con `TextDecoderStream`, separa por línea en blanco y llama a `alEvento()` por cada bloque que empieza con `event:` (comentarios/latidos se ignoran). El evento no trae datos: la vista vuelve a pedir la lista. Un fallo lanza `Error('Eventos: <status>')` y no pasa por el interceptor, de modo que el 401 del flujo no redirige por sí mismo; la vista refresca (por axios), lo que sí dispara el redirect. Tres flujos: `/pagos/eventos`, `/prospectos/eventos`, `/recorridos/eventos`. Patrón de las vistas (README): `AbortController`, bucle, y al caerse refrescar + esperar `ESPERA_RECONEXION_MS` (3000) + reconectar; se aborta al desmontar.
- **Configuración al arrancar:** `App.vue` llama `useConfiguracion().cargar()` (`GET /configuracion`); si falla se queda el diseño original. Aplica color de acento, fuente (carga Google Fonts) y título de la pestaña.
- **Quién llama a la API sin pasar por la vista:** 11 de las 13 vistas importan funciones de api directamente; `LoginView` usa la store auth y `FamiliasView` usa el barrel de clientes y `useConsultaTelefono` (verificado).

5. Componentes, composables y utilidades compartidas

Resultado del refactor (informes `docs/fase*.md`). «Usos» = archivos que lo importan hoy (contados).

Componentes `shared/ui`

Componente	Usos	Qué reemplazó / función
BuscadorInput	7	Campos de búsqueda de Clientes, Prospectos, Pagos, Familias, Asistencias, Casos especiales y ListaEstatusModal. Props: v-model, placeholder, fondo, conFoco, etiquetaLimpiar, limpiarConEsc.
EncabezadoFijo	6	Cabecera pegajosa de Clientes, Prospectos, Pagos, Asistencias, ConductorAlumnos, ConductorAgregar (props capa, relleno).
EncabezadoEscuela	2	Rótulo de escuela en Pagos y Asistencias.
ModalConfirmacion	2	Diálogo de confirmación idéntico de Familias y Casos especiales (Teleport a body, z-40/z-50).
CampoTexto	3	Campos con etiqueta y aviso de Login (2), Familias (4) y Casos especiales (2).
ModalBase	3	Estructura de los modales de detalle de Cliente, Pago y Prospecto **(origen exacto de lo que reemplazó: ver docs/fase2b-modales.md, no revisado aquí)**.

Spinner 18 · DropdownSelect 8 · LoaderVan 7 · SinResultados 4 · SelectorSemana 2 · ServicioEtiqueta 2 · VisorCaptura 2 · BotonScroll 2 · LoaderVanJuego 2 · WhatsappIcon 4 · TelegramIcon 4	Movidos desde `components/` a `shared/ui` en el refactor (no son resultado de deduplicación).

Composables `shared/composables`

Composable	Usos	Qué hace / reemplazó
useDestello<T>(apagado)	3	Devuelve `{resaltado, iluminar}`: resalta 3000 ms (clase `.destello`) lo buscado. Reemplazó estado + temporizador copiados en Clientes, Prospectos y Familias.
useNavegacionConLoader	1	`irA(ruta)`: loader, pausa de 500 ms y `router.push`. Salió de DashboardView.
useFamilias	2	Agrupa por tutor (función pura `utils/familias.agruparPorTutor`) + alumno principal + coincidencia por tutor. Reemplazó la agrupación de Clientes y Prospectos (la de Familias, que ordena por nombre y descarta sin tutor, quedó separada).
useOpcionesEscuela	5	`{opciones, cargar}` para selectores de escuela (Clientes, Prospectos, sus formularios de edición, Registro público).
useCamposTocados	2	`{tocado, tocarTodos, reiniciar}`: avisos por campo en Familias y Casos especiales.
useConsultaTelefono	2	Consulta de teléfono (Familias, Registro público).
useConfiguracion	5	Configuración vigente + aplicación de color/fuente.
useTema	1	Claro/oscuro; `data-theme` en `<html>`; persiste en `localStorage['tema']`.

Utilidades y constantes

- `utils/errores.ts` reemplazó 4 copias de `mensajeDeError` y las comprobaciones de 400/409 dispersas (comparación escrita: 786 casos, 0 diferencias, según docs/fase1).
- `utils/nombres.ts` (`inicialesDeTexto, nombreCompleto, nombreCompletoEstricto, inicialesAlumno`) reemplazó 4+ funciones de iniciales/nombre copiadas. `utils/busqueda.ts` (`normalizar, coincideBusqueda, coincidePorAlumno, filtrarPorAlumnoOTutor`) reemplazó los filtros repetidos de Clientes y Prospectos.
- Otras: `moneda` (`formatearMonto, parseMonto`), `fecha`, `semana` (lunes, rangos, calendario), `orden`, `escuelas` (`colorEscuela`), `servicio`, `telefono`, `tiempo` (`esperar`), `spritesVan`, `familias`.
- Constantes: `MAX_MENSAJE=300`, `LONGITUD_TELEFONO=10`, `ESPERA_RECONEXION_MS=3000`, `DURACION_DESTELLO_MS=3000`, `PAUSA_NAVEGACION_MS=500`, `PAUSA_CARGA_MS=100`.
- Store compartida: `navegacion` (`cargandoPantalla`). API compartida: `http, eventos, escuelas, configuracion, telefono`.

6. Contratos que no deben romperse

Query de la pantalla Pagos (URL ⇄ estado)

`PagosView` lee `route.query` al montarse y, con un `watch`, reescribe la URL con `router.replace({query})` (omite los vacíos). Dashboard enlaza a ella.

Parámetro	Significado y validación
`semana`	Lunes de la semana, `AAAA-MM-DD`; solo se acepta si cumple `^\d{4}-\d{2}-\d{2}$`; si no, semana en curso. Se manda a la API como `GET /pagos/semana?semana=…`.
`escuela`	Filtro de escuela (clave de escuela o clave «sin escuela»).
`estatus`	Solo valores que sean clave de `CLASES_CONTADOR_SELECCIONADO` (tipo `EstatusSemana`); otro valor se ignora.
`q`	Texto de búsqueda (alumno, tutor o teléfonos).
`alumno`	Id de cliente (entero > 0) cuyo detalle se abre.

Quien enlaza hoy: DashboardView — `/pagos?alumno=<id>&semana=<semana>` y `/pagos?estatus=<estatus>&semana=<semana>`. Cambiar nombres o formato rompe esos enlaces y la recarga en el mismo punto.

Claves de `localStorage` (completas, por búsqueda en `src/`)

Clave	Quién la usa	Contenido
`token`	store auth (escribe/borra), `http.ts` (lee y borra en 401), `eventos.ts` (lee)	Token JWT de sesión (no se documenta valor).
`tema`	`useTema`; `index.html` aplica el tema antes de cargar (verificado: un script en index.html lee `tema` antes de cargar la página)	`'dark'` o `'light'`.
`asistencias-escuela`	AsistenciasView	Escuela elegida (cadena; vacía si no hay).

Tokens de diseño (`src/assets/main.css`, Tailwind v4 `@theme`)

- Colores: `--color-bg, panel, panel-2, accent, sun, mint, servicio, van, van-tinta, text-primary, text-secondary, text-muted, danger, border, glass, scroll, scroll-hover`. Radios/espaciado/fuente: `--radius-card` (1.25rem), `--spacing-cabecera` (4rem), `--font-sans` (Manrope).
- **Acento configurable:** `--color-accent: var(--color-accent-user, #ff5c7a)`. `useConfiguracion` escribe `--color-accent-user` (y `--font-sans`) en `<html>`; sin valor se elimina y vuelve el rosa original.
- **Modo claro:** bloque `[data-theme='light']` que redefine bg, panel, sun, mint, servicio, van-tinta, textos, border, glass y scroll. El tema oscuro es el predeterminado.
- Utilidades: `.glass, .glass-plano, .entra-suave` (usa `--i`), `.destello` (animación de 3 s; debe coincidir con `DURACION_DESTELLO_MS`), `.scroll-fino`. Se respeta `prefers-reduced-motion`.

Otros contratos con efecto visible/comportamental documentados en docs/: foco y Esc de los modales (Esc cierra solo con foco dentro; el diálogo de archivar cierra todo el modal de detalle), orden de operaciones de `useDestello`, claves de agrupación (`t-<tutor_id>`, `c-<id>`, `a-<id>`).

7. Cómo se verifica que nada cambia

El refactor a features/shared se hizo sin cambiar comportamiento, con tres redes de seguridad:

1. **Verificador de arquitectura** (§2): `node scripts/verificar-arquitectura.mjs [--detalle] [--estricto]`. Ejecutado hoy: ✓ 0 fallos.
2. **Capturas de regresión visual** (`scripts/capturas.mjs`, Playwright + pixelmatch + pngjs): captura las pantallas en claro/oscuro × móvil/escritorio (la base guardada tiene 140 PNG en `docs/capturas-base/`; 11 pantallas-guion en `PANTALLAS`) y compara con umbral 0.01 por píxel (configurable con `--umbral`). Es de solo lectura: aborta peticiones no GET. Requisitos: la API en marcha (URL de `VITE_API_URL`), puerto 5173 libre (la API solo permite ese origen por CORS, según el comentario del script) y credenciales de admin de prueba en el `.env` del repositorio de la API (se leen en memoria, no se imprimen). Dependen de los datos locales, la fecha y la configuración guardada: si cambian, regrabar la base.
3. **Pruebas de comportamiento** (`scripts/pruebas/`): `dialogos, campos, modales, destello, escuelas, tocados, agrupacion` (+ `agrupacion-unit.mjs`, sin navegador). Cada una mide HEAD y el árbol actual y compara los JSON (`docs/comportamiento/*-head.json` vs `*-arbol.json`). Resultados reportados en docs (no re-ejecutados hoy): dialogos 200 observaciones / 0 difieren; campos 725 / 0; destello 89 / 0; agrupacion 858 / 0; escuelas 60 / 0; tocados 214 / 0; capturas completas 140 imágenes / 0 diferencias.

```
# tipos, build, arquitectura
npx vue-tsc --noEmit
npm run build
node scripts/verificar-arquitectura.mjs --estricto
# linters sin modificar archivos  (npm run lint SÍ aplica --fix)
npx oxlint .   &&   npx eslint . --no-cache
# capturas (API en marcha, puerto 5173 libre)
node scripts/capturas.mjs capturar --salida base --desde-head   # graba base desde HEAD
node scripts/capturas.mjs comparar [--solo login,pagos]         # compara el árbol con la base
node scripts/capturas.mjs diff --a base --b actual
# comportamiento
node scripts/capturas.mjs probar --prueba dialogos --desde-head   # mide HEAD
node scripts/capturas.mjs probar --prueba dialogos                # mide el árbol
node scripts/capturas.mjs comparar-comportamiento --prueba dialogos
node scripts/pruebas/agrupacion-unit.mjs
```

Nota: `docs/comportamiento/` contiene hoy los JSON (head/arbol) de 6 de las 7 pruebas; no existe el archivo `agrupacion-unit.txt` que citan los informes (verificado). `docs/capturas-*/` está en `.gitignore` (verificado, línea 43).

8. Cómo agregar un feature nuevo

1. Crear `src/features/<nombre>/` con las carpetas necesarias (`views/ components/ api/ types/ …`).
2. Escribir `views/<Nombre>View.vue`; lógica en composables, lo repetido en `shared/ui` (solo si hay ≥2 usos reales).
3. Registrar la ruta en `app/router/index.ts`, como hija de `/` (autenticada) o de primer nivel (pública): `component: () => import('@/features/<nombre>/views/<Nombre>View.vue')`, con `path` y `name`.
4. Añadir la entrada de menú en `app/layouts/AppLayout.vue` (arreglo `navItems`).
5. Si importa algo de otro feature: ese feature lo expone en su `index.ts` (solo api/utils/stores/tipos, con nombre) y se añade la arista a `ARISTAS_APROBADAS` en `scripts/arquitectura/mapa.mjs`; si el nuevo feature será importado por otros, añadirlo a `FEATURES_CON_INDEX` y crear su barrel.
6. Las llamadas HTTP van siempre por `shared/api/http.ts` (en `features/<nombre>/api/`); errores con `shared/utils/errores.ts`.
7. Verificar: `vue-tsc`, `build`, verificador y, si procede, capturas.

> **Ojo:** `mapa.mjs` y el verificador aún guardan la lógica de «archivos por mover». Un archivo nuevo dentro de `features/`, `shared/` o `app/` se clasifica por su carpeta sin tocar el mapa (verificado leyendo `info()`); uno fuera de esas zonas y no listado queda «sin zona» y falla.

9. Deuda técnica

Del archivo `docs/deuda-tecnica.md` (cada punto contrastado con el código donde fue posible) y de hallazgos de esta revisión:

#	Deuda	Estado
1	El temporizador de `useDestello` no se limpia al desmontar (conservado a propósito; solo escribe en un ref local).	documentado
2	`irA` de `useNavegacionConLoader` no se cancela al desmontar; un `router.push` pendiente se ejecuta igual.	documentado
3	Casos especiales no reinicia los avisos de campo al elegir otro alumno (Familias sí).	documentado
4	`ClientesView` y `ProspectosView` llaman a `listarEscuelas()` sin `try/catch`.	documentado
5	`verMiRuta` y `terminarRecorrido` (`features/conductores/api/conductores.ts`, vista pública del conductor por token) no tienen consumidores ni ruta en el router: código sin uso aparente.	hallazgo de esta revisión (grep en `src/`)

6	`/apariencia` solo oculta el enlace a no-admin; la ruta no comprueba rol en el cliente (la protección real depende del backend **no verificado**).	hallazgo
7	Token JWT en `localStorage`, sin comprobación de expiración en el cliente (`exp` existe en el tipo pero no se usa); la sesión vencida solo se descubre con un 401.	hallazgo
8	Errores de eslint de la línea base (2, p. ej. un `RouterLink` sin usar en DashboardView) según docs/fase3; estado actual **no verificado** (no se corrió eslint).	documentado
9	No hay pruebas unitarias en el proyecto (ni runner en `package.json`); la verificación es por capturas y scripts ad hoc que requieren API y datos locales.	hallazgo
10	Scripts del refactor (`mapa.mjs` MOVIMIENTOS, `mover.mjs`) siguen en el repositorio aunque la migración terminó (105/105 archivos en su sitio).	hallazgo
11	`docs/deuda-tecnica.md` menciona `REFACTOR_FRONTEND.md`, que no está en la raíz.	documentado

10. Variables de entorno (solo nombres)

Nombre	Dónde	Uso
`VITE_API_URL`	`.env` de este repo (ignorado por git); tipada en `env.d.ts`	URL base del cliente axios y del flujo SSE.
`BASE_URL`	integrada de Vite	Base del historial del router.
`CAPTURAS_RAIZ`	entorno del shell, opcional	Raíz del repo para `scripts/capturas.mjs`.
`ADMIN_CORREO`, `ADMIN_PASSWORD`	`.env` del repositorio de la API (no de este)	Solo las lee `capturas.mjs` para iniciar sesión.

Este documento se generó solo con lectura del repositorio; no se modificó código ni se hicieron commits.